Improving Deep Neural Networks

Unit 4 — Regularization, Optimization & Training Strategies

1 Bias vs. Variance Tradeoff

1.1 Definitions

Bias is the error that arises from incorrect assumptions baked into the learning algorithm itself. A high-bias model is one that is too simple to capture the true underlying patterns in the data. It makes systematic, consistent errors regardless of which training sample it sees.

Variance is the error that arises from excessive sensitivity to the particular training set used. A high-variance model has learned noise and idiosyncrasies specific to the training data and cannot generalise to new, unseen examples. It produces different predictions when trained on different samples drawn from the same distribution.

Intuition. Imagine trying to fit a straight line through data that actually follows a wavy curve. No matter how much data you give the model, the line will always be wrong — that is high bias. Now imagine a model that memorises every single training point perfectly, including the noise; it works great on training data but fails on anything new — that is high variance.

1.2 Underfitting and Overfitting

Underfitting occurs when a model is too simple relative to the complexity of the task. The model has not learned enough from the data and performs poorly even on the training set. Underfitting is the manifestation of high bias.

Overfitting occurs when a model is too complex relative to the amount and noisiness of the data. The model has memorised training examples rather than learning the underlying distribution. It achieves very low training error but high validation error. Overfitting is the manifestation of high variance.

1.3 The Tradeoff

The bias-variance tradeoff is the observation that reducing one source of error tends to increase the other. Increasing model capacity (more layers, more parameters, more expressiveness) reduces bias because the model can represent more complex functions, but it simultaneously increases variance because the model becomes more sensitive to the specific training data. Conversely, reducing model capacity reduces variance but reintroduces bias.

This is not a hard mathematical law in every setting, but it reflects a fundamental tension: a model flexible enough to fit anything will overfit; a model rigid enough to never overfit cannot capture real patterns.

Common misconception. The tradeoff does not mean you must always accept high variance to get low bias or vice versa. Regularization, more data, and better architectures can simultaneously reduce both, moving you toward the lower-left corner of the bias-variance space rather than just trading one for the other.

1.4 Bias-Variance Decomposition of Error

The expected generalisation error of a model can be decomposed as:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- Bias^2: the squared difference between the expected prediction of the model and the true value — the systematic error due to wrong assumptions.
- Variance: the variability of the model's prediction across different training sets drawn from the same distribution.
- **Irreducible Noise**: the inherent randomness in the data-generating process that no model can eliminate.

Intuition. Even a perfect model cannot reduce error below the irreducible noise floor — if the data itself has randomness, the best model still gets that wrong. The goal of training is to squeeze Bias^2 and Variance as low as possible, knowing you can never reach zero total error.

1.5 Diagnosing Bias vs. Variance

The key diagnostic tool is comparing **training error** and **validation error**:

- **High bias (underfitting):** Training error is high. Validation error is also high. The gap between them is small — the model fails everywhere, not just on new data.
- **High variance (overfitting):** Training error is low (sometimes near zero). Validation error is significantly higher. The gap between them is large — the model only works on data it has seen.
- **Good fit:** Training error and validation error are both low, and the gap is small.
- **Both high bias and high variance:** Training error is moderately high, and validation error is even higher. This can happen when a model overfits parts of the training data while systematically underfitting others.

1.6 Practical Fixes

1.6.1 Fixing High Bias

- Use a larger, more expressive model (more layers, more units per layer).
- Train for more epochs — the model may not have converged.
- Reduce regularization strength if regularization is overly aggressive.
- Engineer better features or use a more appropriate architecture for the data type.
- Use a more powerful model class (e.g., switch from a shallow network to a deep one).

1.6.2 Fixing High Variance

- Collect more training data — the most reliable fix.
- Apply regularization (L1, L2, Dropout, Early Stopping).
- Reduce model capacity if it is excessive for the task.
- Use data augmentation to artificially increase training set diversity.
- Apply early stopping to prevent the model from memorising noise.

1.7 Comparison Table: High Bias vs. High Variance

Dimension	High Bias	High Variance
Manifestation	Underfitting	Overfitting
Training error	High	Low (often near zero)
Validation error	High (similar to training)	Much higher than training
Train–val gap	Small	Large
Root cause	Model too simple; wrong assumptions	Model too complex; memorised noise
Primary fix	Increase model capacity; train longer	More data; regularization; reduce capacity
Secondary fix	Better features; better architecture	Data augmentation; dropout; early stopping
Regularization role	Worsens if too strong	Helps; reduces overfitting

2 Saddle Point Problem in Neural Networks

A **saddle point** is a point in the loss landscape where the gradient is zero but which is neither a local minimum nor a local maximum. At a saddle point, the loss curves upward in some directions and downward in others — like the shape of a horse saddle. The gradient being zero makes the optimizer "think" it has found an extremum, but the point is actually unstable.

Intuition. Picture a mountain pass between two peaks. Standing at the pass, going forward takes you downhill, going sideways takes you uphill. The gradient at the exact centre of the pass is zero — you are neither at the top nor the bottom. A simple gradient-based optimizer that stops whenever it sees zero gradient would get stuck here, thinking it found a flat optimum.

2.1 Geometry of Saddle Points

A point θ^* in the parameter space is a saddle point if the Hessian matrix of the loss at θ^* has both positive and negative eigenvalues. In other words, in some directions the loss curves upward (the point is a local minimum along those directions) and in others it curves

downward (the point is a local maximum along those directions). This is in contrast to a true local minimum, where the Hessian is positive semi-definite in every direction.

2.2 Why Saddle Points Dominate in High Dimensions

In a loss landscape with n parameters, each critical point (zero-gradient point) can be classified by asking: in how many of the n directions does the loss curve upward? At a true local minimum, all n directions curve upward. At a saddle point, at least one curves downward.

In high-dimensional spaces, the probability that all directions simultaneously curve upward becomes exponentially small. Statistical physics arguments and empirical evidence both suggest that in deep networks with millions of parameters, the vast majority of critical points are saddle points, not local minima. True local minima are extremely rare.

Important implication. The classical worry in neural network optimisation was getting trapped in local minima. Modern understanding has shifted: the real challenge is saddle points and plateaus. Local minima, when they do exist in deep networks, tend to have loss values close to the global minimum and are therefore not a serious practical problem.

2.3 Why Gradient Descent Gets Stuck

At a saddle point, the gradient is zero (or near zero). Vanilla gradient descent computes updates proportional to the gradient:

$$\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$$

When $\nabla_\theta \mathcal{L} \approx 0$, the update is nearly zero regardless of α. The optimizer effectively halts. Even if the loss could decrease by moving along a downward-curving direction, the near-zero gradient provides no signal to do so. The optimizer lingers at the saddle point for potentially many iterations.

2.4 Why Saddle Points Matter More for Deep Networks

Shallow networks have fewer parameters and lower-dimensional loss landscapes. The proportion of saddle points relative to all critical points grows with dimensionality. Deep networks, having many layers and millions of parameters, live in extremely high-dimensional spaces where saddle points are overwhelmingly common. Additionally, the composition of many non-linear layers creates more complex, non-convex loss landscapes with many more critical points overall.

2.5 How Modern Optimizers Escape Saddle Points

Momentum-based methods accumulate velocity in the parameter space. Even if the gradient momentarily becomes small, the accumulated velocity continues to carry parameters past saddle points.

RMSProp and Adam adapt per-parameter learning rates. When the gradient is consistently small in a direction (as happens near a saddle), these methods may actually increase the effective step size, helping the optimizer escape.

Noise from mini-batching (used in SGD and its variants) introduces stochasticity into the gradient estimate. This noise can push the optimizer off a saddle point even when the true gradient is near zero, because the noisy gradient estimate is unlikely to be exactly

zero.

2.6 Comparison: Saddle Points vs. Local Minima vs. Plateaus

Property	Saddle Point	Local Minimum	Plateau
Gradient	Zero	Zero	Near zero (not exactly)
Loss curvature	Mixed (up in some, down in others)	Upward in all directions	Approximately flat
Hessian	Indefinite (mixed eigenvalues)	Positive semi-definite	Near-zero eigenvalues
Frequency in deep nets	Very common	Rare	Common
Practical danger	Optimizer stalls	Usually not severe (loss near global min)	Slow training; hard to escape
Escape mechanism	Momentum; noise	Not needed often	Adaptive LR; noise

3 Regularization

Regularization refers to any technique that reduces a model's generalisation error (validation/test error) without necessarily reducing training error, typically by constraining, penalising, or perturbing the model to prevent it from memorising the training data. The core problem regularization addresses is overfitting: when a model has enough capacity to fit noise, regularization limits how aggressively it exploits that capacity.

3.1 L1 Regularization (Lasso)

3.1.1 Formulation

L1 regularization adds the sum of the absolute values of the model weights to the loss function:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \lambda \sum_i |w_i|$$

where $\lambda > 0$ is the regularization strength hyperparameter.

3.1.2 Effect on Weights

The L1 penalty creates a constant force that pushes every weight toward zero by an amount λ at each update step (for weights not already at zero). This force is the same regardless of the magnitude of the weight. As a result, small weights are driven all the way to zero, while large weights are merely reduced. This produces **sparsity**: many weights in the final model become exactly zero.

3.1.3 When to Use L1

- When you suspect that only a small number of features or connections are actually relevant.
- When you want automatic feature selection embedded in training (zero weights effectively remove connections).
- When model interpretability benefits from a sparse solution.

Intuition. L1 is like paying a fixed *fee* for every non-zero weight you keep active in the model, regardless of how large that weight is. If a weight contributes little, it is not worth paying the fee, so it gets set to zero. Only weights that genuinely help reduce the loss are worth keeping.

3.2 L2 Regularization (Ridge / Weight Decay)

3.2.1 Formulation

L2 regularization adds the sum of the squared values of the model weights to the loss:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \frac{\lambda}{2} \sum_i w_i^2$$

3.2.2 Effect on Weights

The L2 penalty creates a restoring force proportional to the weight's magnitude: large weights are penalised heavily, small weights barely at all. This results in **weight shrinkage** — all weights are pulled toward zero but typically do not reach it exactly. The model distributes its capacity more evenly across all features/connections rather than concentrating on a few.

3.2.3 When to Use L2

- When you believe most features are relevant and want to prevent any single weight from dominating.
- When you want smooth, distributed representations.
- As the default regularization choice for most deep learning tasks; it is often referred to as *weight decay* in this context.

Intuition. L2 is like paying a fee that scales with the *square* of the weight's size. Very large weights become very expensive; tiny weights cost almost nothing. So the model avoids extreme weights but does not zero anything out entirely.

3.3 L1 vs. L2 Comparison Table

Dimension	**L1 (Lasso)**	**L2 (Ridge)**
Penalty term	$\lambda \sum \lvert w_i \rvert$	$\frac{\lambda}{2} \sum w_i^2$
Gradient of penalty	$\lambda \cdot \text{sign}(w_i)$ (constant force)	λw_i (proportional force)

Dimension	L1 (Lasso)	L2 (Ridge)
Effect on weights	Drives weights exactly to zero (sparsity)	Shrinks weights toward zero, rarely exactly zero
Sparsity	Yes — produces sparse solutions	No — all weights remain non-zero
Feature selection	Implicit — zero weights remove features	No implicit selection
Preferred when	Few relevant features; need sparsity/interpretability	Most features relevant; want distributed capacity
Sensitivity to outliers	More robust (linear penalty)	Less robust (squared penalty amplifies large errors)

3.4 Batch Normalization

Batch Normalization (BatchNorm) is a technique that normalizes the activations of each layer across the mini-batch during training. For a given layer, it transforms the pre-activation values so that they have zero mean and unit variance within the mini-batch, then applies a learned affine rescaling.

3.4.1 Mechanism

For a mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$ at a given layer, BatchNorm computes:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the mini-batch mean and variance, ϵ is a small constant for numerical stability, and γ and β are **learnable parameters** that allow the network to undo the normalization if needed.

3.4.2 Why It Helps

Internal covariate shift refers to the phenomenon where the distribution of each layer's inputs changes during training as the parameters of all preceding layers change. This forces later layers to continuously adapt to a moving target, slowing convergence. BatchNorm reduces this shift by re-centring and re-scaling activations after each mini-batch.

Additional benefits:

- **Smoother loss landscape:** Normalized activations produce more stable gradients, allowing higher learning rates without divergence.
- **Implicit regularization:** The noise introduced by computing statistics over a mini-batch (rather than the full dataset) acts as a regularizer, reducing the need for dropout in some architectures.
- **Reduces sensitivity to weight initialization:** The network starts in a better-conditioned regime regardless of initial weight scales.

3.4.3 Learnable Parameters γ and β

After normalization, the distribution is forced to have mean 0 and variance 1, which could destroy information (e.g., if the network had learned that a particular layer's activations should be large). The learnable scale parameter γ and shift parameter β restore the network's ability to represent any mean and variance it needs, while still benefiting from the improved gradient flow.

3.4.4 Placement in the Network

BatchNorm is typically placed **after the linear transformation and before the activation function** (i.e., between the weight matrix multiplication and the ReLU). Some practitioners place it after the activation; this is an architectural choice with minor differences in practice. It is applied to each layer independently.

3.4.5 Training vs. Inference Behaviour

During **training**, the mean and variance are computed from the current mini-batch. This introduces noise that acts as regularization.

During **inference**, the model needs deterministic output regardless of batch size. BatchNorm therefore uses **running estimates** of the population mean and variance (maintained as exponentially decaying averages of the per-batch statistics during training) rather than batch statistics. This makes the behaviour at inference time deterministic and independent of batch size.

> **Exam trap.** BatchNorm behaves differently at training time and inference time. During training it normalizes per mini-batch; during inference it uses fixed running statistics. Forgetting this distinction is a common error. Additionally, setting batch size to 1 at inference is fine, but using batch size 1 during training makes BatchNorm's statistics meaningless because variance across a single sample is undefined.

3.4.6 Advantages and Disadvantages

Advantages:

- Accelerates training by stabilizing gradient flow.
- Allows use of higher learning rates.
- Reduces sensitivity to initialization.
- Acts as a mild regularizer.

Disadvantages:

- Adds computational overhead (computing batch statistics at every layer).
- Behaves poorly with very small batch sizes (statistics become noisy).
- Introduces a dependency between samples in the same mini-batch, which breaks the independence assumption in some settings (e.g., online learning, certain RNN tasks).
- Less effective for recurrent networks — Layer Normalization is preferred there.

3.5 Early Stopping

Early stopping is a regularization strategy in which training is halted before the model fully converges on the training set. Training is monitored via a held-out validation set, and training is stopped when the validation loss stops improving (or begins to worsen), preventing the model from overfitting.

3.5.1 How It Works

During training, the validation loss is evaluated at regular intervals (e.g., once per epoch). A counter called **patience** tracks how many consecutive evaluations have passed without improvement. If the validation loss does not improve by more than a minimum threshold over p consecutive evaluations, training is stopped and the model weights from the best-performing checkpoint are restored.

3.5.2 Why It Acts as Regularization

Early stopping implicitly constrains the model to a region of weight space reachable in a limited number of gradient steps from the initialization. This limits the effective complexity of the final model — not by changing the architecture, but by stopping before the model has had time to overfit. This is analogous to L2 regularization in certain theoretical frameworks.

3.5.3 Practical Considerations

- **Patience:** A patience value that is too small causes premature stopping during temporary fluctuations in validation loss. A value that is too large provides little benefit over training to full convergence.
- **Checkpoint saving:** The weights at the best validation loss checkpoint should be saved; simply stopping at the last step may yield worse weights than an earlier epoch.
- **Risk of stopping too early:** Some models exhibit a "double descent" phenomenon or delayed generalization, where the validation loss temporarily worsens before improving again. Stopping during such a phase discards potentially better performance.

Intuition. Training a network is like cooking a dish: you check the taste periodically (validation loss). If the dish tastes best at 20 minutes but you keep cooking until 60 minutes, it burns (overfits). Early stopping is the equivalent of removing it from the heat at the optimal moment and not waiting for the timer to ring.

3.6 Dropout

Dropout is a regularization technique in which, during each training step, each neuron in a designated layer is independently and randomly deactivated (set to zero) with probability p (the **dropout rate**). The surviving neurons are scaled up by $\frac{1}{1-p}$ to maintain the expected activation magnitude.

3.6.1 Mechanism

At each forward pass during training, a binary mask is sampled independently for each neuron. A neuron is active with probability $(1-p)$ and dropped with probability p. The gradient is not propagated through dropped neurons. This changes the effective network architecture at each step, making the network train on an exponential number of different sub-networks that share weights.

3.6.2 Why It Prevents Co-Adaptation

Without dropout, neurons can develop **co-adaptation**: some neurons become highly specialized to correct for the mistakes of others. This tight coupling reduces robustness. With dropout, no neuron can rely on the presence of any other neuron (since any might be dropped), so each neuron must learn features that are independently useful. This forces more distributed, redundant, and robust representations.

3.6.3 Ensemble Interpretation

Dropout can be interpreted as training an implicit ensemble of 2^n sub-networks (where n is the number of dropout-eligible neurons). At inference, instead of running all 2^n sub-networks and averaging their predictions (which would be computationally intractable), the full network with all neurons active is used, with each weight scaled by $(1 - p)$. This approximates the ensemble average.

3.6.4 Training vs. Inference Behaviour

During **training**, neurons are randomly dropped with probability p, and the remaining activations are scaled up by $\frac{1}{1-p}$ (inverted dropout).

During **inference**, dropout is disabled — all neurons are active, and no scaling is applied (since the weights were already scaled during training). This ensures deterministic, full-capacity predictions at test time.

> **Exam trap.** Dropout is only active during training. During inference, the full network is used without any random deactivation. Some implementations forget to disable dropout at inference time, causing non-deterministic and degraded test performance. This is a well-known implementation error.

3.6.5 Advantages

- Effective regularizer that significantly reduces overfitting.
- Encourages redundant, distributed feature learning.
- Computationally cheap to implement.
- Provides implicit ensemble averaging.

3.6.6 Disadvantages

- Doubles or triples the number of training epochs needed to converge (since many neurons are disabled per step).
- High dropout rates ($p > 0.5$) can significantly impair training and may lead to high bias.
- Not well-suited to very small networks where there are few neurons to spare.

3.6.7 When Not to Use Dropout

- When using Batch Normalization heavily — the two interact in complex ways and BN already provides implicit regularization.
- In convolutional layers with spatial dropout not applied — standard dropout on individual pixels is less effective in CNNs; spatial dropout (dropping entire feature maps) is preferred.

- When the model is already underfitting (high bias) — dropout will only worsen the situation by further reducing effective capacity.

3.7 Pruning

Pruning is the process of removing weights, neurons, attention heads, or entire layers from a trained model to produce a smaller, more efficient model. Unlike most regularization techniques applied during training, pruning is typically applied *post-training* (or iteratively during fine-tuning).

3.7.1 Motivation

Deep networks are often massively over-parameterised. After training, many weights are close to zero and contribute minimally to the model's predictions. Pruning removes these redundant components to reduce memory footprint, inference latency, and energy consumption — making models suitable for deployment on edge devices or in latency-sensitive applications.

3.7.2 Structured vs. Unstructured Pruning

Unstructured pruning removes individual weights (sets them to zero) regardless of their position in the weight matrix. This produces sparse weight matrices. While it can achieve high compression ratios, sparse matrix operations do not always translate to speedups on standard hardware without dedicated sparse computation support.

Structured pruning removes entire neurons, filters, attention heads, or layers. The resulting model has a smaller *dense* architecture, which maps directly to reduced computation on standard hardware without requiring any sparse computation libraries.

3.7.3 Trade-offs

- Aggressive pruning degrades model accuracy; a fine-tuning phase after pruning is typically needed to recover performance.
- The optimal pruning criterion (what to remove) is non-trivial; simple magnitude-based criteria (remove smallest weights) are effective in practice.
- Iterative pruning — prune a small fraction, fine-tune, repeat — generally outperforms one-shot pruning.

3.8 Regularization Comparison Table

Technique	Mechanism	Effect on Model	When to Use	Main Cost
L1 (Lasso)	Penalty: $\lambda \sum \lvert w_i \rvert$; constant shrink force	Sparse weights; feature selection	Sparse features expected	Computationally cheap
L2 (Ridge)	Penalty: $\frac{\lambda}{2} \sum w_i^2$; proportional shrink force	Weight shrinkage; distributed capacity	Default for most DL tasks	Computationally cheap

Technique	Mechanism	Effect on Model	When to Use	Main Cost
Batch Norm	Normalises activations per mini-batch; learns γ, β	Stable gradient flow; faster training	Deep networks; large batches	Extra compute; poor with small batches
Early Stopping	Halts training when val. loss stops improving	Prevents over-training; limits effective capacity	Any task with a held-out val. set	Must monitor carefully; premature stop risk
Dropout	Randomly deactivates neurons per training step	Redundant, independent features; ensemble effect	Fully connected layers; moderate overfitting	Slower convergence; more epochs needed
Pruning	Removes weights/neurons post-training	Smaller, faster model	Deployment on edge / latency-sensitive apps	Fine-tuning needed; accuracy drop risk

4 Optimization Techniques

An **optimizer** is the algorithm that updates the model's parameters during training to minimize the loss function. The optimizer determines how the gradient information is used to compute each parameter update — the step direction, the step size, and how past gradient information is incorporated.

4.1 Batch Gradient Descent

In **Batch Gradient Descent (BGD)**, the gradient of the loss is computed over the entire training dataset before any parameter update is made:

$$\theta \leftarrow \theta - \alpha \cdot \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathcal{L}(x_i, y_i; \theta)$$

where N is the total number of training samples.

Characteristics:

- The gradient estimate is exact (no sampling noise), producing smooth, stable convergence.
- Each update requires a full pass over all training data — extremely slow for large datasets.
- Memory requirements scale with dataset size, making it infeasible for modern large-scale training.

- Deterministic: given the same initialization, it always follows the same path.
- Not suitable for online learning (cannot update as new data arrives).

4.2 Stochastic Gradient Descent (SGD)

In **Stochastic Gradient Descent**, the gradient is computed on a single randomly selected training example per update:

$$\theta \leftarrow \theta - \alpha \cdot \nabla_\theta \mathcal{L}(x_i, y_i; \theta)$$

Characteristics:

- Very fast updates; the model sees many updates per epoch.
- Gradient estimate is very noisy (high variance), causing erratic convergence paths.
- The noise can actually help escape saddle points and shallow local minima.
- Convergence is unstable: the loss oscillates heavily rather than smoothly decreasing.
- Memory efficient: only one sample needs to be in memory at a time.
- Poorly suited to parallelization (each update depends on the previous).

4.3 Mini-Batch Gradient Descent

Mini-Batch Gradient Descent computes the gradient over a small, fixed subset (mini-batch) of the training data:

$$\theta \leftarrow \theta - \alpha \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}(x_i, y_i; \theta)$$

where B is the batch size (typically 32, 64, 128, or 256).

Mini-batch GD is the practical default in deep learning because it balances the stability of BGD and the speed of SGD. Typical batch sizes of 32–256 provide gradient estimates with manageable noise while being efficiently parallelisable on GPU hardware. It is often loosely referred to simply as "SGD" in the literature, though this is technically imprecise.

Intuition. Instead of asking all $N = 1{,}000{,}000$ employees their opinions before making one decision (BGD), or asking a single random employee (pure SGD), you ask a random committee of 64 employees. The committee gives you a good-enough estimate quickly, and you can form many such committees in the time it would take to poll everyone.

4.4 Momentum-Based Gradient Descent

Momentum augments gradient descent by accumulating a velocity vector in parameter space. Instead of updating directly in the direction of the current gradient, the update blends the current gradient with the history of past gradients.

4.4.1 Update Rule

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_\theta \mathcal{L}_t$$

$$\theta \leftarrow \theta - \alpha v_t$$

where v_t is the velocity at step t, $\beta \in [0, 1)$ is the momentum coefficient (typically 0.9), and α is the learning rate. (Some formulations omit the $(1 - \beta)$ factor and absorb it into α; both conventions are common.)

4.4.2 Why It Helps

- **Dampens oscillations:** In directions where the gradient repeatedly changes sign (e.g., across a narrow valley), the positive and negative velocity components cancel, reducing the oscillation.
- **Accelerates in consistent directions:** In directions where the gradient consistently points the same way, velocity accumulates and the effective step size grows.
- **Escapes saddle points:** Accumulated velocity can carry the optimizer past saddle points where the instantaneous gradient is near zero.

Intuition. Imagine rolling a ball down a hilly surface. The ball does not stop instantly when it reaches flat ground; its velocity carries it forward. Momentum in gradient descent works the same way: past gradient information gives the optimizer "inertia" that helps it glide through flat regions and past small obstacles.

4.5 RMSProp

RMSProp (Root Mean Square Propagation) adapts the learning rate for each parameter independently by dividing by the root of an exponentially decaying average of the squared gradients.

4.5.1 Update Rule

$$s_t = \rho s_{t-1} + (1 - \rho) \left(\nabla_\theta \mathcal{L}_t\right)^2$$
$$\theta \leftarrow \theta - \frac{\alpha}{\sqrt{s_t + \epsilon}} \nabla_\theta \mathcal{L}_t$$

where s_t is the exponentially decaying average of squared gradients, ρ is the decay rate (typically 0.9), α is the global learning rate, and ϵ is a small stabilising constant (e.g., 10^{-8}).

4.5.2 Why It Helps

Parameters whose gradients have been consistently large receive a smaller effective learning rate (large s_t divides the step). Parameters whose gradients have been consistently small receive a larger effective learning rate (small s_t divides the step less). This automatic per-parameter learning rate adaptation is particularly beneficial for:

- Non-stationary objectives (loss surfaces that shift over time).
- Recurrent neural networks (RNNs), where gradients can vary widely across time steps and parameters.
- Problems with sparse gradients.

Difference from AdaGrad. RMSProp was developed specifically to address AdaGrad's problem of monotonically decreasing learning rates (where accumulated squared gradients only grow, making the effective learning rate shrink to zero). RMSProp's exponential decay forgets old gradient information, preventing the learning rate from collapsing.

4.6 Adam Optimizer

Adam (Adaptive Moment Estimation) combines momentum (first-order moment estimation) and RMSProp (second-order moment estimation) with bias correction to produce an optimizer that adapts per-parameter learning rates while maintaining momentum.

4.6.1 Update Rule

First moment (mean of gradients):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla_\theta \mathcal{L}_t$$

Second moment (uncentred variance of gradients):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\left(\nabla_\theta \mathcal{L}_t\right)^2$$

Bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter update:

$$\theta \leftarrow \theta - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

4.6.2 Hyperparameters

- β_1: Decay rate for the first moment (default: 0.9). Controls the momentum of the mean gradient.
- β_2: Decay rate for the second moment (default: 0.999). Controls the adaptation of the per-parameter learning rate.
- ϵ: Small constant for numerical stability (default: 10^{-8}). Prevents division by zero.
- α: Global learning rate (typically 0.001).

4.6.3 Bias Correction

In the first few steps, m_t and v_t are initialised at zero and have not yet accumulated enough history to be reliable estimates. Bias correction compensates for this initialisation bias, ensuring that the early steps are not taken with a disproportionately small effective learning rate.

4.6.4 Why Adam is the Default Optimizer

Adam combines the benefits of momentum (smooth convergence, saddle point escape) and adaptive learning rates (robust to learning rate choice, good for sparse/non-stationary

gradients). It is robust to a wide range of hyperparameter settings, converges quickly in practice, and performs well across a broad variety of architectures and tasks. This makes it the pragmatic default in most deep learning frameworks.

Common misconception: Adam is not always best. In certain tasks — particularly image classification with well-tuned architectures (ResNets, etc.) — SGD with momentum and a carefully tuned learning rate schedule has been shown to generalise better than Adam. Adam can converge to sharper minima that generalise less well than the broader minima found by SGD with momentum. When generalisation matters more than convergence speed, and when you have the compute budget to tune SGD carefully, SGD with momentum can outperform Adam.

Intuition. Adam is like a smart GPS: it adapts the speed (learning rate) for each parameter based on both recent gradient history (momentum) and how steep the road has been (RMSProp). Vanilla SGD is like walking with a fixed stride length — it might take longer and wobble, but sometimes the slower, more careful walker ends up in a better spot (broader minimum with better generalisation).

4.7 Comparison Table: All Optimizers

Optimizer	Update Rule (Summary)	Adaptive LR?	Convergence	Memory	When to Use
Batch GD	$\theta -= \alpha \overline{\nabla \mathcal{L}}$ (full dataset)	No	Smooth, stable	Low	Small datasets; convex problems
SGD	$\theta -= \alpha \nabla \mathcal{L}$ (1 sample)	No	Noisy, unstable	Minimal	Large datasets; noise beneficial
Mini-Batch GD	$\theta -= \alpha \overline{\nabla \mathcal{L}}$ (mini-batch)	No	Moderate; practical default	Low	Standard DL; GPU-friendly
Momentum	Accumulates velocity v_t; dampens oscillations	No	Smoother than SGD	$+1\times$ (velocity)	Narrow valleys; saddle points
RMSProp	Adapts LR via $s_t = \rho s_{t-1} + (1-\rho)g^2$	Yes (per-param)	Stable; non-stationary	$+1\times$ (s_t)	RNNs; non-stationary objectives
Adam	m_t (momentum) + v_t (RMSProp) + bias correction	Yes (per-param)	Fast; robust to LR	$+2\times$ (m_t, v_t)	Default; transformers; CNNs